

Trade-offs of custom agents

Why workflow agents are preferred

Factor	Workflow agents	Custom agents
Complexity	Simple, declarative	More code to write
Maintenance	Maintained by ADK	You maintain it
Testing	Standard patterns	Custom test cases
Debugging	Clear execution flow	More complex flow

The rule of thumb

Default to workflow agents. Only use custom agents when you’ve confirmed that sequential, parallel, and loop cannot handle your specific conditional logic requirements.

What custom agents look like (preview)

For awareness, here’s the basic structure of a custom agent:

```
Python
from google.adk.agents import BaseAgent
from google.adk.events import Event

class ConditionalRouter(BaseAgent):
    """Custom agent with if/else logic"""

    async def _run_async_impl(self, ctx):
        # Read condition from state
        request_type = ctx.session.state.get('request_type')
```

```
# Conditional routing
if request_type == 'billing':
    async for event in self.billing_agent.run_async(ctx):
        yield event
else:
    async for event in self.general_agent.run_async(ctx):
        yield event
```

Note: This is just a preview. Full implementation patterns are covered in the **advanced learning path**.

Key takeaways

Most multi-agent workflows can be built using the standard workflow agents. Custom agents are powerful but add complexity—only use them when you’ve confirmed that workflow agents genuinely cannot solve your problem.

Workflow agents first

- SequentialAgent, ParallelAgent, and LoopAgent cover most use cases
- Nest workflow agents for complex patterns
- Default to these before considering custom agents

Custom agents are rare

- Needed for conditional skipping or dynamic routing
- More complex to implement and maintain
- Only use when workflow agents truly can’t work

Learning path

Your Need	Where to Learn
Basic workflows	Lesson 7, lesson 8 M1-M2
Custom conditional logic	Advanced learning path
Production patterns	Advanced learning path